

Практическое занятие 31: Введение в Django REST Framework, создание API

Цель занятия:

- Понять концепцию RESTful API и ее использование.
 - Научиться устанавливать и настраивать Django REST Framework (DRF) в проекте Django.
 - Создать API для вашего приложения с использованием DRF.
 - Научиться создавать сериализаторы и представления для обработки данных.
 - Научиться тестировать и использовать созданные API-эндпоинты.
-

Задачи занятия

1. Введение в RESTful API и Django REST Framework
 2. Установка и настройка Django REST Framework
 3. Создание сериализаторов для моделей
 4. Создание API-представлений
 5. Настройка URL-маршрутизации для API
 6. Тестирование API-эндпоинтов
-

Задача 1: Введение в RESTful API и Django REST Framework

1.1. Что такое RESTful API?

- **REST (Representational State Transfer)** — архитектурный стиль взаимодействия клиент-сервер, использующий стандартные HTTP-методы.
- **RESTful API** позволяет обмениваться данными между клиентом и сервером в формате JSON или XML.
- **Преимущества RESTful API:**
 - Легкость и простота использования.
 - Кроссплатформенность.
 - Стандартные методы HTTP (GET, POST, PUT, DELETE).

1.2. Что такое Django REST Framework?

- **Django REST Framework (DRF)** — мощный и гибкий инструмент для построения веб-API на основе Django.
 - **Особенности DRF:**
 - Простые и мощные сериализаторы данных.
 - Гибкие представления (Views) для обработки запросов.
 - Поддержка аутентификации и разрешений.
 - Возможность генерации интерактивной документации API.
-

Задача 2: Установка и настройка Django REST Framework

2.1. Установка DRF

Шаги:

1. Установите Django REST Framework с помощью pip:

```
pip install djangorestframework
```

2. Убедитесь, что установка прошла успешно:

```
pip show djangorestframework
```

2.2. Настройка проекта для использования DRF

Шаги:

1. Добавьте 'rest_framework' в список `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'rest_framework',
    'events', # Ваше приложение
]
```

2. (Опционально) Настройте глобальные настройки DRF:

```
# settings.py
```

```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ],
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
        'rest_framework.authentication.BasicAuthentication',
    ],
}
```

Пояснение:

- Здесь мы устанавливаем разрешения по умолчанию (`AllowAny`), чтобы любой пользователь мог получить доступ к API.
- Вы можете настроить аутентификацию и разрешения позже, в зависимости от требований вашего проекта.

Задача 3: Создание сериализаторов для моделей

3.1. Что такое сериализаторы?

- **Сериализаторы** преобразуют сложные типы данных, такие как объекты моделей Django, в форматы данных, которые можно легко преобразовать в JSON или XML.
- **Сериализаторы** также обеспечивают валидацию входящих данных при создании или обновлении объектов.

3.2. Создание сериализатора для модели `Event`

Шаги:

1. Создайте файл `serializers.py` в приложении `events`:

```
touch events/serializers.py
```

2. Откройте `serializers.py` и добавьте следующий код:

```
# events/serializers.py

from rest_framework import serializers
from .models import Event

class EventSerializer(serializers.ModelSerializer):
    class Meta:
```

```
model = Event
fields = '__all__'
```

Пояснение:

- Мы создаем класс `EventSerializer`, который наследуется от `serializers.ModelSerializer`.
- Поле `fields = '__all__'` означает, что мы хотим включить все поля модели `Event`.

3. Создайте сериализатор для модели `Review` (если необходимо):

```
# events/serializers.py

from .models import Review

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = '__all__'
```

Задача 4: Создание API-представлений

4.1. Использование функциональных представлений API

Шаги:

1. Откройте `views.py` в приложении `events`.
2. Импортируйте необходимые модули:

```
# events/views.py

from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .serializers import EventSerializer
from .models import Event
```

3. Создайте представление для получения списка мероприятий:

```
# events/views.py

@api_view(['GET', 'POST'])
def api_event_list(request):
```

```

if request.method == 'GET':
    events = Event.objects.all()
    serializer = EventSerializer(events, many=True)
    return Response(serializer.data)
elif request.method == 'POST':
    serializer = EventSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data,
status=status.HTTP_201_CREATED)
    return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

```

Пояснение:

- Декоратор `@api_view` определяет, какие методы HTTP поддерживаются (GET, POST).
- При GET запросе возвращаем список всех мероприятий.
- При POST запросе создаем новое мероприятие после валидации данных.

4. Создайте представление для получения, обновления и удаления конкретного мероприятия:

```

# events/views.py

@api_view(['GET', 'PUT', 'DELETE'])
def api_event_detail(request, pk):
    try:
        event = Event.objects.get(pk=pk)
    except Event.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = EventSerializer(event)
        return Response(serializer.data)
    elif request.method == 'PUT':
        serializer = EventSerializer(event, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
    elif request.method == 'DELETE':
        event.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

Пояснение:

- Обработываем `GET`, `PUT` и `DELETE` запросы для работы с конкретным объектом `Event`.

4.2. Использование классов-представлений API (опционально)

Шаги:

1. Импортируйте классы-представления:

```
# events/views.py

from rest_framework import generics
```

2. Создайте класс-представление для списка мероприятий:

```
# events/views.py

class EventListAPIView(generics.ListCreateAPIView):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

3. Создайте класс-представление для деталей мероприятия:

```
# events/views.py

class EventDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

Пояснение:

- Классы-представления позволяют сократить код и предоставить более гибкую функциональность.

Задача 5: Настройка URL-маршрутизации для API

5.1. Добавление маршрутов API в `urls.py`

Шаги:

1. Откройте `urls.py` в приложении `events`.
2. Импортируйте представления API:

```
# events/urls.py

from . import views
```

3. Добавьте URL-маршруты для API:

Если вы используете функциональные представления:

```
# events/urls.py

urlpatterns += [
    path('api/events/', views.api_event_list, name='api_event_list'),
    path('api/events/<int:pk>', views.api_event_detail,
name='api_event_detail'),
]
```

Если вы используете классы-представления:

```
# events/urls.py

from django.urls import path
from .views import EventListAPIView, EventDetailAPIView

urlpatterns += [
    path('api/events/', EventListAPIView.as_view(),
name='api_event_list'),
    path('api/events/<int:pk>', EventDetailAPIView.as_view(),
name='api_event_detail'),
]
```

5.2. (Опционально) Использование маршрутизаторов DRF

Шаги:

1. Импортируйте `routers` из DRF:

```
# events/urls.py

from rest_framework import routers
```

2. Создайте маршрутизатор и зарегистрируйте представления:

```
# events/urls.py
```

```
router = routers.DefaultRouter()
router.register(r'api/events', views.EventViewSet)
```

3. Включите маршруты маршрутизатора:

```
# events/urls.py

urlpatterns += router.urls
```

4. Создайте EventViewSet в views.py:

```
# events/views.py

from rest_framework import viewsets

class EventViewSet(viewsets.ModelViewSet):
    queryset = Event.objects.all()
    serializer_class = EventSerializer
```

Пояснение:

- Маршрутизаторы и ViewSets позволяют автоматизировать создание URL-маршрутов и упростить код представлений.

Задача 6: Тестирование API-эндпоинтов

6.1. Использование браузера для тестирования

- Перейдите по URL: <http://127.0.0.1:8000/api/events/>
- Вы должны увидеть список мероприятий в формате JSON.
- DRF предоставляет веб-интерфейс для взаимодействия с API через браузер.

6.2. Использование инструмента `curl`

Пример получения списка мероприятий:

```
curl -X GET http://127.0.0.1:8000/api/events/
```

Пример создания нового мероприятия:

```
curl -X POST -H "Content-Type: application/json" -d '{"title": "Новое мероприятие", "description": "Описание", "date": "2023-10-31"}'
http://127.0.0.1:8000/api/events/
```


6.3. Использование Postman или Insomnia

- **Postman** и **Insomnia** — популярные инструменты для тестирования API.
 - **Шаги:**
 - Установите Postman или Insomnia.
 - Создайте новый запрос к вашему API.
 - Тестируйте различные методы (GET, POST, PUT, DELETE).
-

Дополнительные задания

1. Добавление аутентификации и разрешений

Цель: Ограничить доступ к API только для аутентифицированных пользователей.

Шаги:

1. Настройте разрешения в `settings.py`:

```
# settings.py

REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

2. Добавьте маршруты для аутентификации в `urls.py`:

```
# events/urls.py

from rest_framework.auth_token import views as drf_views

urlpatterns += [
    path('api-token-auth/', drf_views.obtain_auth_token,
        name='api_token_auth'),
]
```

3. Установите `rest_framework.auth_token`:

```
pip install django-rest-framework-auth-token
```

4. Добавьте `'rest_framework.auth_token'` в `INSTALLED_APPS`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'rest_framework.authtoken',
]
```

5. Выполните миграции:

```
python manage.py migrate
```

6. Используйте токены для аутентификации при запросах к API.

2. Создание сериализаторов и представлений для модели

Review

- Создайте `ReviewSerializer` и соответствующие представления для работы с отзывами через API.
- Настройте вложенные сериализаторы, чтобы при получении мероприятия также получать связанные отзывы.

Заключение

Поздравляем! Вы успешно:

- Установили и настроили Django REST Framework в своем проекте.
- Создали сериализаторы для моделей.
- Реализовали API-представления для обработки запросов.
- Настроили маршрутизацию URL для API.
- Протестировали API-эндпоинты с помощью браузера и инструментов тестирования.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- **Каковы преимущества использования Django REST Framework по сравнению с обычными представлениями Django?**
 - DRF предоставляет готовые инструменты для создания API, такие как сериализаторы, ViewSets и маршрутизаторы, что упрощает и ускоряет разработку. Он также поддерживает гибкие настройки аутентификации, разрешений и пагинации, что делает его более мощным и удобным для работы с RESTful API по сравнению с традиционными представлениями Django.
 - **Как DRF помогает в реализации RESTful API?**
 - DRF предоставляет структуру и компоненты, специально разработанные для создания RESTful API, включая сериализаторы для преобразования данных, ViewSets для обработки CRUD операций и маршрутизаторы для автоматической генерации URL. Это позволяет разработчикам легко создавать, управлять и масштабировать API, следуя принципам REST.
 - **Какие методы аутентификации поддерживает DRF?**
 - DRF поддерживает различные методы аутентификации, включая базовую аутентификацию, токен-аутентификацию, аутентификацию на основе сессий, JWT (JSON Web Tokens) и OAuth. Это позволяет выбрать наиболее подходящий метод для конкретных требований безопасности и удобства использования вашего API.
-

Полезные ресурсы

- [Документация Django REST Framework](#)
- [Tutorial: Django REST Framework](#)
- [Видео-туториал по Django REST Framework](#)
- [Демонстрация использования DRF](#)